



PasswordStore Protocol Audit Report

Version 1.0

Audited by:

LAU LOK JING

April 25, 2026

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
 - High
 - * [H-1] Passwords stored on-chain are readable by everyone
 - * [H-2] `PasswordStore::setPassword` lacks access control, anyone can change the password
 - Informational
 - * [I-1] The `PasswordStore::getPassword` natspec indicates a parameter that doesn't exist, causing the natspec to be incorrect

Protocol Summary

PasswordStore Protocol is a smart contract applicatoin for storing a password. The protocol is designed for single user application. Each user should be able to store a password and then retrieve it later. Others should not be able to access the password or set it.

Disclaimer

I, Lau Lok Jing makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

The findings described in this document correspond to the following commit hash:

```
1 2e8f81e263b3a9d18fab4fb5c46805ffc10a9990
```

Github Repository: github.com/Cyfrin/3-passwordstore-audit/

Scope

```
1 ./src/  
2 #-- PasswordStore.sol
```

- Solc Version: 0.8.18
- Chain(s) to deploy contract to: Ethereum

Roles

- Owner: The user who can set the password and read the password.
- Outsiders: No one else should be able to set or read the password.

Executive Summary

About 3 hours are spent on the findings using fuzz tests through Foundry. A total of 3 risks are found which composed of 2 high and 1 informational. Specifically, the two high risks found require the user to rethink on the protocol implementation.

Issues found

Severity	Number of issues found
High	2
Medium	0
Low	0
Info	1
Gas Optimizations	0
Total	3

would require the user to remember another password off-chain to decrypt the password. However, you'd also likely want to remove the view function as you wouldn't want the user to accidentally send a transaction with the password that decrypts your password.

[H-2] PasswordStore::setPassword lacks access control, anyone can change the password

Description: The `PasswordStore::setPassword` does not check for the identity of the contract caller, `msg.sender`. However, the natspec of the function describe its purpose as, and quote: *This function allows only the owner to set a new password..*

Impact: Anyone can set/change password of the contract, which impaired the main objective of the contract - User should be able to retrieve the initial configured password.

Proof of Concept: Add the following test to the `PasswordStore.t.sol` file:

Code

```
1     function test_anyone_can_set_password(address _addr) public {
2         if (_addr == owner) return;
3
4         vm.prank(_addr);
5         string memory expectedPassword = "myNewPassword";
6         passwordStore.setPassword(myNewPassword);
7
8         vm.prank(owner);
9         assertEquals(passwordStore.getPassword(), expectedPassword);
10    }
```

Recommended Mitigation: Add an access control conditional to the `passwordStore::setPassword` function.

```
1     if (msg.sender != s_owner) {
2         revert PasswordStore__NotOwner();
3     }
```

Informational

[I-1] The PasswordStore::getPassword natspec indicates a parameter that doesn't exist, causing the natspec to be incorrect

Description:

```
1      /*
2      * @notice This allows only the owner to retrieve the password.
3  @>   * @param newPassword The new password to set.
4      */
5      function getPassword() external view returns (string memory) {
```

The natspec for the function `PasswordStore::getPassword` indicates it should have a parameter with the signature `getPassword(string)`. However, the actual function signature is `getPassword()`.